

Session 5

DOM Attributes, Styling & Array Methods

JavaScript Browser Interaction + Array Manipulation

HTML element updates, image switching, attributes, CSS styles and core array operations

SESSION 5 FOCUS

Learn how JavaScript can change what users see in the browser, manipulate element attributes and styles, and transform arrays with common built-in methods.

Area	What you learn
DOM styling	Change CSS through element.style and nested properties
Attributes	Read, set, check and remove HTML attributes
Browser interaction	Use click handlers to update an image dynamically
Arrays	Add, remove, replace, combine, reverse and sort values
Iteration	Use forEach() and related array methods to process values
Revision	20 practice problems + answer key + mini projects

DESIGN NOTE

This booklet keeps the clean Session 4 visual language with a macOS-style sans-serif text face, dark code blocks, blue explanation cards, green result cards and a clickable index.

Table of Contents

Clickable index - select a topic to jump directly to its page

Page	Topic
04	Session 5 Roadmap
05	DOM Element Selection
06	innerHTML
07	style Property
08	style.backgroundColor + fontSize
09	setAttribute()
10	getAttribute()
11	hasAttribute() + removeAttribute()
12	Image Attribute Switching
13	onclick Event Handler
14	Array Fundamentals
15	push()
16	pop()
17	shift()
18	unshift()
19	shift() vs unshift()
20	splice()
21	splice() - Add
22	splice() - Delete
23	splice() - Replace
24	join()
25	reverse()
26	sort()
27	Array Indexing + Second-Last Element

NAVIGATION

Every topic entry is clickable. Use Return to Index at the bottom of every page to return here.

[Return to Index](#)

Table of Contents - Continued

Clickable index - continued

Page	Topic
28	slice() + concat()
29	includes(), indexOf(), lastIndexOf()
30	forEach()
31	map()
32	filter()
33	find() + findIndex()
34	some() + every()
35	Source Execution Walkthrough
36	DOM + Array Connection
37	Practice 1-10
38	Practice 11-20
39	Answer Key 1-10
40	Answer Key 11-20
41	Mini Project - Image Switcher
42	Mini Project - Array Manager
43	Interview Revision
44	Quick Revision Sheet
45	Session 5 Summary
46	Source Corrections & Clarifications
47	Clean Source HTML - Part 1
48	Clean Source HTML - Part 2
49	Clean Source HTML - Part 3
50	Clean Source JavaScript - Embedded

INDEX CONTINUATION

Second half of the clickable Session 5 index.

[Return to Index](#)

Session 5 Roadmap

How DOM updates and arrays connect

```
JavaScript Session 5
|
+-- DOM interaction
|   +-- select element
|   +-- change style / HTML
|   +-- read or set attributes
|   +-- update image on click
|
+-- Array manipulation
    +-- add -> remove -> insert -> replace
    +-- join -> reverse -> sort
    +-- access by index -> process with callbacks
```

CORE IDEA

The DOM part changes browser state. The array part changes or analyzes collections of values. Both rely on selecting data, performing an operation, and inspecting the result.

DOM Element Selection

Start by selecting the element you want to change

Most DOM updates begin with selecting an element from the document. The source uses `getElementById()` to find the image before changing its style and src attribute.

```
const image = document.getElementById("img");  
console.log(image);  
console.log(image.src);
```

MENTAL MODEL

document -> find element -> receive Element object -> read or modify its properties.

SOURCE CONNECTION

The supplied HTML gives the image the id="img", which is then selected from JavaScript.

innerHTML

Insert or replace HTML inside an element

The commented source demonstrates innerHTML as a way to place HTML markup inside the #main element.

```
const sDiv = document.getElementById("main");
sDiv.innerHTML = "<div id=hey>Hey</div>";
```

WHAT IT DOES

innerHTML parses the supplied string as HTML and replaces the element's current HTML contents with the parsed markup.

```
sDiv.innerHTML = "<div id=hey>Hey</div>";
// Result inside #main:
// <div id="hey">Hey</div>
```

CAUTION

Use innerHTML carefully when inserting untrusted text because HTML parsing can create injection risks. For plain text, `textContent` is safer.

style Property

Change CSS directly from JavaScript

Every HTML element exposes a style object that lets JavaScript set inline CSS properties.

```
const sDiv = document.getElementById("main");
sDiv.style.color = "cyan";
sDiv.style.borderLeft = "5px solid cyan";
sDiv.style.borderRight = "5px solid purple";
sDiv.style.borderBottom = "5px solid beige";
sDiv.style.borderTop = "5px solid lightblue";
```

CSS PROPERTY NAMING

JavaScript style properties use camelCase: backgroundColor, fontSize, borderLeft, marginTop and so on.

SOURCE CONNECTION

The source also uses innerDiv.style.backgroundColor and innerDiv.style.fontSize on the nested #hey element.

style.backgroundColor + fontSize

Update specific CSS properties

The source shows styling a nested element after inserting it into the DOM.

```
const innerDiv = document.getElementById("hey");
innerDiv.style.backgroundColor = "purple";
innerDiv.style.fontSize = "35px";
```

EXPECTED EFFECT

The #hey element receives a purple background and a larger 35px font size.

IMPORTANT

Direct style changes create inline styles. For reusable designs, CSS classes are usually easier to maintain; `classList` is the natural next step.

setAttribute()

Create or change an HTML attribute

setAttribute(name, value) assigns an attribute on an element. It is the method used by the source to swap the image src.

```
const image = document.getElementById("img");  
image.setAttribute("src", "new-image-url");
```

SOURCE CONNECTION

The `change()` and `reverse()` functions both call `setAttribute("src", ...)`, which replaces the image resource.

RULE

Attribute values are strings. The method can set existing attributes or create an attribute that was not previously present.

getAttribute()

Read an attribute value

getAttribute(name) returns the current string value of an element attribute, or null when the attribute is absent.

```
const image = document.getElementById("img");
console.log(image.getAttribute("src"));
console.log(image.getAttribute("alt"));
```

WHY IT MATTERS

Use `getAttribute()` when you specifically need the attribute value as written on the element.

SOURCE NOTE

The source lists `getAttribute` and `setAttribute` as concepts, but only `setAttribute` is executed. This page documents the missing read operation.

hasAttribute() + removeAttribute()

Check or remove an attribute

These are natural companion methods to `getAttribute()` and `setAttribute()`.

```
const image = document.getElementById("img");
console.log(image.hasAttribute("src"));
image.removeAttribute("alt");
console.log(image.hasAttribute("alt"));
```

RESULT

`hasAttribute()` returns true or false. `removeAttribute()` removes the named attribute from the element.

Image Attribute Switching

Change the image source dynamically

The session uses a button to replace the image source with another URL. The same idea can be reused for galleries, theme images and status indicators.

```
function change() {
  const image = document.getElementById("img");
  image.style.transition = "1s linear";
  image.setAttribute("src", "new-image-url");
}

function reverse() {
  const image = document.getElementById("img");
  image.style.transition = "1s linear";
  image.setAttribute("src", "original-image-url");
}
```

FLOW

Click button -> run function -> select image -> set transition -> replace src -> browser loads the new image.

onclick Event Handler

Connect a button click to JavaScript

The supplied HTML uses inline onclick attributes to call `change()` and `reverse()`.

```
<button id="btn" onclick="change()">Click to change</button>  
<img id="img" ...>  
<button id="reverse" onclick="reverse()">Reverse</button>
```

SOURCE CONNECTION

`onclick="change()"` calls the `change` function when the user clicks the button. The reverse button does the same for `reverse()`.

EXTENSION

A later style of event handling is `addEventListener("click", handler)`, which keeps JavaScript behavior separate from HTML markup.

Array Fundamentals

Arrays as mutable ordered collections

The source creates a list of numbers and then applies multiple methods to it. Array indexes start at 0, and most mutating methods change the original array.

```
const li = [11, 22, 33, 44, 55, 66];  
console.log(li[0]); // 11  
console.log(li.length); // 6
```

Category	Methods in Session 5
Add	push(), unshift()
Remove	pop(), shift()
Insert / replace	splice()
Convert / rearrange	join(), reverse(), sort()
Access / iteration	indexing, length, forEach()

SOURCE CORRECTION

The supplied code uses `li = [...]` without `const/let/var`. Use `const li = [...]` or `let li = [...]` so the variable is declared explicitly.

push()

Array methods

Adds one or more elements to the end of an array. It returns the new array length.

WHAT IT DOES

Use `push()` when new values should be appended to the end. It mutates the original array.

```
const li = [11,22,33,44,55,66];  
console.log(li.push(345,333));  
console.log(li);
```

EXPECTED RESULT

8 [11,22,33,44,55,66,345,333]

pop()

Array methods

Removes the last element from an array and returns the removed value.

WHAT IT DOES

pop() is useful when you need both the updated array and the removed item.

```
const li = [11,22,33,44,55,66];  
console.log(li.pop());  
console.log(li);
```

EXPECTED RESULT

66 [11,22,33,44,55]

shift()

Array methods

Removes the first element from an array and returns that removed value.

WHAT IT DOES

shift() is the front-removal partner of pop().

```
const li = [11,22,33,44,55,66];
const removed = li.shift();
console.log(removed);
console.log(li);
```

EXPECTED RESULT

11 [22,33,44,55,66]

SOURCE / COMMON PITFALL

The source comment says "Get the first removed value using unshift()". That is not correct: shift() returns the removed first value; unshift() adds values to the front.

unshift()

Array methods

Adds one or more elements to the beginning of an array and returns the new array length.

WHAT IT DOES

unshift() adds values. With no arguments, it makes no content change and simply returns the current length.

```
const li = [22,33,44];  
console.log(li.unshift(11));  
console.log(li);
```

EXPECTED RESULT

4 [11,22,33,44]

SOURCE / COMMON PITFALL

The supplied source calls li.unshift() with no values while describing replacement/getting removed values. That description is incorrect.

shift() vs unshift()

Front-of-array operations

Method	Action	Returns
shift()	Remove first element	Removed element
unshift(value)	Add at beginning	New array length

```
const li = [22, 33, 44];
const removed = li.shift();
li.unshift(11);
console.log(removed); // 22
console.log(li);      // [11,33,44]
```

MEMORY TRICK

shift = take away from the front. unshift = put onto the front.

splice()

Insert, delete and replace in one method

splice(start, deleteCount, ...items) changes the original array by removing, inserting or replacing elements.

```
li.splice(start, deleteCount, item1, item2, ...);
```

Goal	Pattern
Insert	splice(index, 0, value)
Delete	splice(index, count)
Replace	splice(index, count, value)

IMPORTANT

splice() mutates the original array and returns an array containing the removed elements.

splice() - Add

Array methods

Insert a value without deleting existing elements by using deleteCount = 0.

WHAT IT DOES

The inserted item appears at index 1 and the existing items shift to the right.

```
const li = [22,33,44,55];  
li.splice(1,0,888);  
console.log(li);
```

EXPECTED RESULT

```
[22,888,33,44,55]
```

splice() - Delete

Array methods

Delete one or more elements by giving a start index and deleteCount.

WHAT IT DOES

deleteCount tells JavaScript how many elements to remove starting from the chosen index.

```
const li = [22,888,33,44,55];  
const removed = li.splice(1,4);  
console.log(removed);  
console.log(li);
```

EXPECTED RESULT

[888,33,44,55] [22]

splice() - Replace

Array methods

Replace existing elements by deleting them and supplying replacement values in the same call.

WHAT IT DOES

Replace = start at an index -> remove count -> insert replacement values.

```
const li = [22,66,345];  
li.splice(1,1,345);  
console.log(li);
```

EXPECTED RESULT

[22,345,345]

join()

Convert array values into one string

join(separator) combines array elements into a string. The separator is inserted between each element.

```
const li = [22,345,345];  
const str = li.join("_");  
console.log(str);
```

EXPECTED RESULT

22_345_345

NOTE

join() does not mutate the array. It returns a new string value.

reverse()

Reverse array order

reverse() reverses the elements of the array in place and returns the same array object.

```
const li = [22,345,345];  
const rev = li.reverse();  
console.log(rev);  
console.log(li);
```

EXPECTED RESULT

[345,345,22] [345,345,22]

IMPORTANT

reverse() mutates the original array. If the original order must be preserved, copy first, for example ``const copy = [...li]``.

sort()

Sort array elements

sort() sorts an array in place. By default, JavaScript compares values as strings, so numeric arrays need a comparator for numeric ascending order.

```
const sortArray = [10, 2, 30, 4];  
console.log(sortArray.sort());  
console.log(sortArray.sort((a, b) => a - b));
```

EXPECTED RESULT

Default: [10, 2, 30, 4] can behave lexicographically. Numeric comparator: [2, 4, 10, 30]

SOURCE CONNECTION

The supplied source creates a string array named sortArray but does not call sort(). This page completes the method concept and shows the important numeric-sort rule.

Array Indexing + Second-Last Element

Access values without a method call

The source asks how to access the second-last element. Use `length - 2` as the index because the final index is `length - 1`.

```
const li = [345,345,22];  
console.log(li[li.length - 2]);
```

EXPECTED RESULT

345

```
Index of last item      = li.length - 1  
Index of second-last   = li.length - 2
```

REMEMBER

Indexing starts at 0, so length and last index are always different numbers.

slice() + concat()

Useful non-mutating array companions

Method	Purpose	Mutates?
slice(start,end)	Copies a portion of an array	No
concat(values)	Combines arrays/values into a new array	No

```
const li = [11,22,33,44];  
console.log(li.slice(1,3));  
console.log(li.concat([55,66]));
```

WHY ADD THESE

They are natural non-mutating counterparts to splice() and are useful when you want to read or combine values without changing the original array.

includes(), indexOf(), lastIndexOf()

Find values in arrays

These methods answer whether a value exists and, when needed, where it occurs.

```
const li = [11,22,33,22];  
console.log(li.includes(22));  
console.log(li.indexOf(22));  
console.log(li.lastIndexOf(22));
```

EXPECTED RESULT

true 1 3

PATTERN

includes() -> yes/no. indexOf() -> first index. lastIndexOf() -> last index.

forEach()

Run code for every array element

forEach(callback) visits each element in order and runs the callback once for each existing element. It is primarily used for side effects such as logging or updating the DOM.

```
const values = [10, 20, 30];
values.forEach((value, index) => {
  console.log(index, value);
});
```

CALLBACK DATA

The callback commonly receives (value, index, array).

IMPORTANT

forEach() does not build a new transformed array. Use map() when the goal is to create a new array of transformed values.

map()

Create a transformed array

map(callback) returns a new array containing the callback result for each element.

```
const values = [10,20,30];  
const doubled = values.map(value => value * 2);  
console.log(doubled);
```

EXPECTED RESULT

[20,40,60]

RULE

map() is for transformation: one input element -> one output element.

filter()

Keep only elements that pass a condition

filter(callback) returns a new array containing only the elements for which the callback returns true.

```
const values = [10,15,20,25];  
const even = values.filter(value => value % 2 === 0);  
console.log(even);
```

EXPECTED RESULT

[10,20]

find() + findIndex()

Find the first matching element

Method	Returns
find(predicate)	First matching element, or undefined
findIndex(predicate)	Index of first match, or -1

```
const values = [10,15,20,25];  
console.log(values.find(v => v > 18));  
console.log(values.findIndex(v => v > 18));
```

EXPECTED RESULT

20 2

some() + every()

Test array conditions

Method	Question
some(predicate)	Does at least one element pass?
every(predicate)	Do all elements pass?

```
const values = [2,4,6];  
console.log(values.some(v => v > 5));  
console.log(values.every(v => v % 2 === 0));
```

EXPECTED RESULT

true true

CONNECTION TO SESSION 2

These methods return Boolean results and fit naturally with the logical/comparison concepts from Session 2.

Source Execution Walkthrough

What happens to li step by step

Step	Operation	Array after operation
Start	const li = [11,22,33,44,55,66]	[11,22,33,44,55,66]
1	push(345,333)	[11,22,33,44,55,66,345,333]
2	pop()	[11,22,33,44,55,66,345]
3	shift()	[22,33,44,55,66,345]
4	unshift() with no args	[22,33,44,55,66,345]
5	unshift() with no args again	[22,33,44,55,66,345]
6	splice(1,0,888)	[22,888,33,44,55,66,345]
7	splice(1,4)	[22,66,345]
8	splice(1,1,345)	[22,345,345]
9	join("_")	"22_345_345"
10	reverse()	[345,345,22]

KEY CORRECTION

The two unshift() calls in the source do not add or replace anything because no values were provided. They simply return the current array length.

DOM + Array Connection

A practical browser pattern

```
const items = document.querySelectorAll("li");  
const names = Array.from(items).map(item => item.textContent);  
console.log(names);
```

CONCEPT

DOM selection can produce collections; array methods can then process those values. This is a natural bridge from Session 5 fundamentals toward interactive web interfaces.

Practice 1-10

DOM attributes and style exercises

Attempt each problem before checking the answer key.

#	Problem
1	Select the image by id and print its src attribute.
2	Use <code>setAttribute()</code> to replace an image src.
3	Use <code>getAttribute()</code> to read alt text.
4	Check whether an image has a src attribute using <code>hasAttribute()</code> .
5	Remove the alt attribute with <code>removeAttribute()</code> .
6	Change an element background color using <code>style.backgroundColor</code> .
7	Change font size and <code>borderLeft</code> through style.
8	Use <code>innerHTML</code> to insert a small child div.
9	Explain the flow of <code>onclick="change()"</code> .
10	Build a button that changes an image and another that restores it.

Practice 11-20

Array methods and indexing exercises

Use only methods covered in Session 5 plus basic JavaScript syntax.

#	Problem
11	Add three values to the end of an array using push().
12	Remove the last value and store it in a variable using pop().
13	Remove the first value and display the removed item using shift().
14	Add two values to the beginning using unshift().
15	Insert a value at index 2 using splice().
16	Delete two values beginning at index 1 using splice().
17	Replace one element using splice().
18	Convert an array to a string using join("_").
19	Reverse an array and then access its second-last element.
20	Sort a numeric array in ascending order correctly.

Answer Key 1-10

Suggested solutions

```
1. const src = image.getAttribute("src");
2. image.setAttribute("src", newUrl);
3. console.log(image.getAttribute("alt"));
4. console.log(image.hasAttribute("src"));
5. image.removeAttribute("alt");
6. element.style.backgroundColor = "purple";
7. element.style.fontSize = "35px"; element.style.borderLeft = "5px solid cyan";
8. element.innerHTML = "<div>Hey</div>";
9. Browser click -> onclick -> change() -> select image -> set src.
10. Use two functions and setAttribute("src", url) in each.
```

NOTE

These are compact reference answers. Expand them into full programs while practicing.

Answer Key 11-20

Suggested solutions

```
11. arr.push(10, 20, 30);
12. const removed = arr.pop();
13. const first = arr.shift();
14. arr.unshift(10, 20);
15. arr.splice(2, 0, value);
16. arr.splice(1, 2);
17. arr.splice(index, 1, replacement);
18. const result = arr.join("_");
19. arr.reverse(); console.log(arr[arr.length - 2]);
20. arr.sort((a, b) => a - b);
```

MEMORY RULE

push/pop work at the end; shift/unshift work at the beginning; splice handles index-based edits; join returns a string; reverse and sort mutate the array.

Mini Project - Image Switcher

Combine DOM selection, style and attributes

Build a two-button image switcher based on the supplied `change()` and `reverse()` functions.

Requirement	Concept
Select image	<code>getElementById()</code>
Change image	<code>setAttribute("src", ...)</code>
Smooth transition	<code>style.transition</code>
Reverse image	second handler restores original src
User interaction	<code>onclick</code> or <code>addEventListener()</code>

TARGET

Click Change -> new image appears. Click Reverse -> original image returns. Keep the image id unique and use valid image URLs.

Mini Project - Array Manager

Practice the complete array method family

Create a small array manager that lets you add, remove, insert, replace, join and sort values.

```
const values = [11,22,33,44,55];  
// add -> push()  
// remove end -> pop()  
// remove front -> shift()  
// add front -> unshift()  
// insert / delete / replace -> splice()  
// string -> join()  
// reverse -> reverse()  
// numeric order -> sort((a,b) => a-b)
```

EXTENSION

Display the array in the DOM after every operation instead of only using console.log().

Interview Revision

Fast answers for Session 5

Question	Answer
What does push() return?	The new array length.
What does pop() return?	The removed last element.
shift() vs unshift()?	Remove first vs add at beginning.
What does splice() do?	Insert, delete and replace elements at a chosen index.
Does join() mutate the array?	No, it returns a string.
Does reverse() mutate?	Yes. It reverses the original array.
Why use a comparator with sort()?	Default sort is lexicographic; a-b gives numeric ascending order.
setAttribute()?	Sets or replaces an attribute value.
getAttribute()?	Reads an attribute value.
innerHTML?	Parses HTML and replaces an element's contents.
onclick?	Runs a handler when the element is clicked.

INTERVIEW TIP

Define the method first, then state whether it mutates the array or returns a new value. That distinction is often more useful than memorizing examples.

Quick Revision Sheet

Session 5 at a glance

```
DOM
getElementById -> select one element by id
innerHTML -> replace/insert HTML content
style.* -> set inline CSS properties
setAttribute -> set/change attribute
getAttribute -> read attribute
hasAttribute -> test attribute existence
removeAttribute -> remove attribute
onclick -> run function after click

ARRAYS
push -> add end
pop -> remove end
shift -> remove front
unshift -> add front
splice -> insert/delete/replace
join -> array to string
reverse -> reverse in place
sort -> sort in place
slice / concat -> non-mutating copies/combinations
forEach -> process each item
map -> transform
filter -> keep matches
find / findIndex -> first match
some / every -> Boolean tests
```

MEMORY MAP

DOM -> select -> read/change. Arrays -> add/remove/edit -> rearrange -> process.

Session 5 Summary

What you should be able to do now

After Session 5, you should be able to:

Skill	Expected ability
DOM selection	Select elements by id and work with element objects
HTML content	Insert or replace markup with innerHTML
Inline styling	Set CSS properties through element.style
Attributes	Read, set, check and remove attributes
Events	Connect clicks to JavaScript functions
Array mutation	Use push, pop, shift, unshift and splice
Array conversion	Use join to create a string
Array rearrangement	Reverse and numerically sort values
Array access	Use length and index arithmetic
Array processing	Use forEach, map, filter, find and condition tests

NEXT STEP

These operations become much more powerful when combined with DOM rendering, event listeners, loops and reusable functions.

Source Corrections & Clarifications

Important notes from the supplied code

Source item	Clarification
li = [..]	Declare the variable with const or let.
li.unshift()	With no arguments, it does not change the array; it returns the current length.
"Get the first removed value using unshift()"	The removed first value comes from shift(), not unshift().
"To replace the first" + unshift()	unshift() does not replace; splice() or direct index assignment can replace.
sortArray created but not sorted	A sort() call was missing; numeric sorting needs (a,b) => a-b.
reverse()	Mutates the original array.
setAttribute(src)	Changes the image resource if the URL is valid.

WHY THIS PAGE EXISTS

The source contains useful experiments and comments, but several comments do not match the actual behavior of the method. This page keeps the original intent while making the final study notes technically usable.

Clean Source HTML - Part 1

Appendix: original Session 5 HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    #main{
      width: 400px;
      height: 400px;
      background-color: grey;
      margin: 50px auto;
    }

    #hey{
      background-color: rgb(179, 122, 122);
    }
  </style>
</head>
<body>
  <!-- <div id="main">
  </div> -->
  <button id="btn" onclick="change()">Click to change</button>
  
  <button id="reverse" onclick="reverse()">Reverse</button>

  <script>
    // let sDiv = document.getElementById('main')
    // sDiv.innerHTML = "<div id=hey> Hey </div>"
    // sDiv.style.color = "cyan"
    // sDiv.style.borderLeft = "5px solid cyan"
    // sDiv.style.borderRight = "5px solid purple"
    // sDiv.style.borderBottom = "5px solid beige"
    // sDiv.style.borderTop = "5px solid lightblue"

    // let innerDiv = document.getElementById('hey')
    // innerDiv.style.backgroundColor = "purple"
    // innerDiv.style.fontSize = "35px"

    //getAttribute

    //setAttribute

    function change(){
      let image = document.getElementById('img')
      image.style.transition = "1s linear"
      image.setAttribute('src', 'https://media.istockphoto.com/id/1171712711/photo/supernova-
explosion-in-space.jpg?s=612x612&w=0&k=20&c=7fwusg4bEZ47MWHKTudqAr6DZu3Dh0fkowlPr16d39E=')
    }
  </script>
</body>
</html>
```

Clean Source HTML - Part 2

Appendix: original Session 5 HTML

```
function reverse(){
    let image = document.getElementById('img')
    image.style.transition = "1s linear"
    image.setAttribute('src','https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcQ40AugpFARNot9ly2idC95xS6nXIYRISzIJYtTdoJDPw&s=10')
}

// Array Methods

li = [11,22,33,44,55,66]

li.push(345,333)
console.log("Added two values using push()", li)

li.pop()
console.log("Deleted last value using pop()", li)

li.shift()
console.log("Removed first value using shift()", li)

li.unshift()
console.log("Get the first removed value using unshift()", li)

li.unshift()
console.log("To replace the first", li)

// SPLICE

// add
li.splice(1,0,888)
console.log("Added using splice: " , li)

// delete
li.splice(1,4)
console.log("delete using splice: " , li)

// replace
li.splice(1,1,345)
console.log("Replaced using splice: " , li)

// JOIN method

const str = li.join('_')
console.log("Join using _ => ", str)
```

Clean Source HTML - Part 3

Appendix: original Session 5 HTML

```
// Reverse()

const rev = li.reverse()
console.log("Reverse => ", rev)

// sort()

const sortArray = ["1", '2', '3', '4', '5', '6']

// Q) access the second last element from the array

console.log(li[li.length-2])


// For each method use different array list
</script>
</body>
</html>
```

Clean Source JavaScript - Embedded

Appendix note

SOURCE NOTE

The supplied Session 5 JavaScript is embedded inside the HTML file. The complete original HTML appendix preserves that code exactly as supplied.